

KOGI PLATFORM

kogi-engine

Data & Intelligence Engine

Design Document — v1.0 · March 2026

kogi-engine is the central data-processing, analytics, and intelligence platform of the Kogi system. It consumes all platform event streams, runs multi-dimensional analytics, executes graph traversals on the portfolio dependency graph, powers personalized recommendations and content delivery, drives worker and resource matching, optimizes system resource allocation, interprets and optimizes SQL queries, and continuously assesses portfolio risk and health. Every sub-engine exposes a unified interface through the KogiEngine facade, accessible via gRPC server, REST HTTP server, and interactive CLI.

Package	kogi.engine
Language	Scala 3 (primary) · compiled on JVM
Interfaces	gRPC (EngineGrpcServer) · HTTP REST (GraphEngineAPIServer) · CLI (KogiEngineCli, GraphEngineCLI)
Sub-engines	Analytics · Telemetry · Recommendation · Personalization · Match · Graph · Risk · Optimization · Search · Query · DataStreaming
Persistence	In-memory (production swap: PostgreSQL / Kafka / Redis)
Entry point	Main.scala · EngineGrpcServer · GraphEngineAPIServer
Status	MVP — all sub-engines implemented and integrated

1. SYSTEM ARCHITECTURE

1.1 Design Philosophy

kogi-engine is built around a single organizing principle: all data that flows through the Kogi platform — user interactions, module telemetry, portfolio events, host metrics, gateway messages — is ingested, processed, and made actionable through one unified engine. The engine never owns user-facing product logic; it provides the intelligence substrate that all other modules (Office, Bank, Exchange, Community, Marketplace) consume.

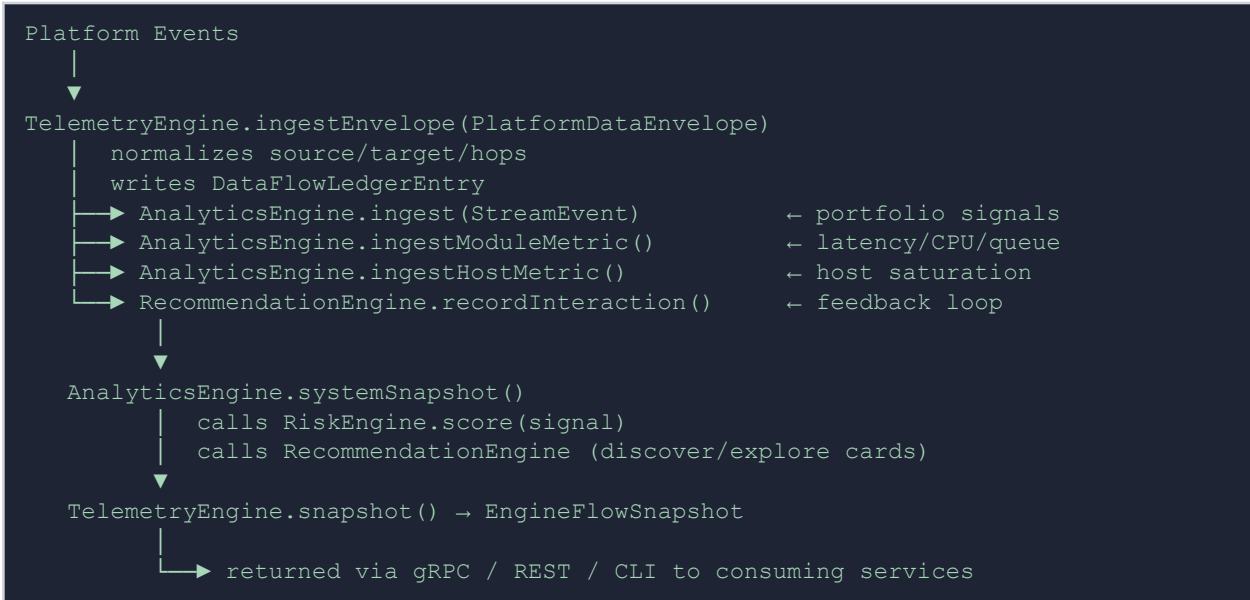
The architecture is layered: raw events enter through the TelemetryEngine, which normalizes and routes them to the AnalyticsEngine and the RecommendationEngine's feedback loop. The AnalyticsEngine derives portfolio health signals and module-level metrics, which feed the RiskEngine and OptimizationEngine. The GraphEngine operates independently on the portfolio dependency graph. All of these are exposed through the KogiEngine facade and served via gRPC, REST, and CLI interfaces.

1.2 Sub-engine Map

Sub-engine	File	Core Responsibility
KogiEngine	KogiEngine.scala	Unified facade — single access point for all sub-engines
AnalyticsEngine	AnalyticsEngine.scala	Event ingestion, module metrics, host metrics, portfolio signals, system snapshots, recommendations/discover/explore
TelemetryEngine	TelemetryEngine.scala	Platform data envelope processing, flow ledger, component flow stats, snapshot generation
DataStreamingEngine	DataStreamingEngine.scala	In-memory event stream, pub/sub subscriptions, profile/module/time-window queries
RecommendationEngine	RecommendationEngine.scala	User profiling, collaborative/content-based/hybrid/contextual/cold-start recommendations, feedback loop
PersonalizationEngine	PersonalizationEngine.scala	Content delivery adaptation, user segmentation, A/B experiments, multi-armed bandit, persona lifecycle
MatchEngine	MatchEngine.scala	Multi-dimensional matching of users, portfolio components, resources, analytics artifacts
GraphEngine	GraphEngine.scala	Dependency graph traversal, cycle detection, topological sort, critical path (CPM), graph diff
RiskEngine	RiskEngine.scala	Portfolio health scoring, persona-aware risk adjustment, risk optimization plans
OptimizationEngine	OptimizationEngine.scala	System resource optimization recommendations, persona-aware workload tuning
SearchEngine	SearchEngine.scala	Full-text + tag/metadata indexing, personalized re-ranking
QueryEngine	QueryEngine.scala	SQL normalization, cost estimation, query optimization, persona-aware row caps

1.3 Data Flow

The primary data path through kogi-engine:



1.4 Interface Layer

Interface	File	Protocol	Key Commands / Endpoints
gRPC Server	EngineGrpcServer.scala	gRPC / protobuf (google.Struct)	Control · Status · Ingest · Snapshot
HTTP REST Server	GraphEngineAPIServer.scala	HTTP/1.1 JSON (JDK HttpServer)	GET /health · POST /load · GET /nodes · GET /edges · GET /impact · GET /topo · GET /critical · GET /cycles · GET /diff
Engine CLI	KogiEngineCli.scala	stdin/stdout (JSON output)	control · status · ingest · snapshot · query · search · graph-impact · graph-cycles · graph-critical · graph-topo · graph-diff
Graph CLI	GraphEngineCLI.scala	stdin/stdout (text + JSON + DOT)	impact · closure · rdeps · topo · critical · cycles · neighbors · ancestors · descendants · diff · nodes · edges · export
Programmatic API	GraphEngineAPIFacade.scala	Scala in-process	Builder API, typed Either results, snapshot/diff, mutation methods

2. KogiEngine — Unified Facade

2.1 Purpose

KogiEngine is the single entry point for all engine functionality. It owns and composes every sub-engine instance, provides lifecycle control (start / pause / stop), and exposes delegation methods for every sub-engine capability under a single object reference. Consumers of kogi-engine — whether a Go service, a gRPC client, or another Scala module — interact exclusively through KogiEngine.

2.2 Construction & Wiring

```
new KogiEngine(  
  recommendationEngine = new RecommendationEngine(),  
  riskEngine           = new RiskEngine(),  
  analyticsEngineOverride = null,           // auto-wired: AnalyticsEngine(risk, rec)  
  telemetryEngineOverride = null,          // auto-wired: TelemetryEngine(analytics)  
  optimizationEngine     = new OptimizationEngine(),  
  searchEngine           = new SearchEngine(),  
  queryEngine            = new QueryEngine(),  
  matchEngine            = new MatchEngine(),  
  initialGraphEdges      = Seq.empty,  
  initialGraphNodes      = Seq.empty  
)
```

The AnalyticsEngine and TelemetryEngine are auto-wired so they share the same RiskEngine and RecommendationEngine instances, ensuring consistent signal propagation without duplicate computation.

2.3 Lifecycle Control

Action	Mode Transition	Notes
control("start") / control("run") / control("resume")	→ running	Idempotent
control("pause")	→ paused	Sub-engines continue holding state
control("stop") / control("shutdown")	→ stopped	Graceful; state preserved in memory
status	(read-only)	Returns current mode, changed flag, timestamp

2.4 GraphEngine Lifecycle

KogiEngine maintains a mutable GraphEngine instance. Edges and nodes can be added, removed, or bulk-loaded atomically. Each mutation rebuilds the internal GraphEngine from scratch (immutable Seq rebuild).

Method	Description
--------	-------------

graphAddEdge(from, to, edgeType, weight)	Add a single directed edge; rebuilds graph
graphAddNode(id, duration)	Add a node with optional CPM duration; rebuilds graph
graphRemoveEdge(from, to)	Remove edge by endpoints; rebuilds graph
graphRemoveNode(id)	Remove node and all incident edges; rebuilds graph
graphLoad(edges, nodes)	Atomic full replacement of the graph
graphSnapshot()	Returns an immutable copy for later diffing
graphDiff(snapshot)	Differs current live graph against an earlier snapshot

3. AnalyticsEngine

3.1 Purpose

The AnalyticsEngine is the real-time analytics and observability core of kogi-engine. It ingests three kinds of data — platform stream events, module performance metrics, and host infrastructure metrics — and produces per-profile analytics snapshots, per-module realtime snapshots, and system-level snapshots combining host health, module health, portfolio signals, and AI-driven recommendation/discover/explore cards.

3.2 Data Model

Input Types

Type	Key Fields	Source
StreamEvent	profileId · module · eventType · timestampMs · productivityDelta · cashFlowDelta · collaborationDelta · riskDelta	All platform modules via TelemetryEngine
ModuleMetricSample	module · latencyMs · queueDepth · errorCount · successCount · throughputUnits · cpuPct · memoryMb	Module services (latency/queue/cpu readings)
HostMetricSample	hostId · cpuPct · memoryPct · diskPct · processCount · schedulerLoad · networkIn/OutKbps	kogi-host infrastructure layer

Output Types

Type	Contents
AnalyticsSnapshot	profileId · eventCount · PortfolioSignal · PortfolioHealthScore · module activities · recommendation/discover/explore cards
ModuleRealtimeSnapshot	Per-module: throughput/min · error rate · p95 latency · queue depth · CPU/memory averages · health status · anomalies · AI cards
HostRealtimeSnapshot	hostId · CPU/memory/disk/process averages · saturation score (weighted) · status (green/amber/red) · anomalies
SystemRealtimeSnapshot	Aggregates HostRealtimeSnapshot + List[ModuleRealtimeSnapshot] + system-level AI recommendation/discover/explore cards

3.3 Portfolio Signal

The AnalyticsEngine aggregates StreamEvents into a PortfolioSignal — a four-dimensional score representing the health of a user's or module's work portfolio:

Dimension	Source	Meaning
productivity	productivityDelta fields, throughput metrics	Work output rate and task completion velocity

cashFlow	cashFlowDelta fields, financial module events	Financial health — inflows vs. outflows
collaboration	collaborationDelta fields, community/chat events	Cross-user and cross-team coordination quality
risk	riskDelta fields, error rates, anomaly counts	Exposure to failure, overload, or instability

3.4 Anomaly Detection

The AnalyticsEngine applies threshold-based anomaly detection to module and host metrics:

- Module: latency p95 > 2000ms → "high latency" anomaly
- Module: error rate > 10% → "error spike" anomaly
- Module: queue depth > 200 → "queue overload" anomaly
- Host: CPU > 85% → "cpu critical" anomaly
- Host: memory > 90% → "memory critical" anomaly
- Host: disk > 95% → "disk critical" anomaly
- Host: saturation score > 0.8 → "host saturation critical" anomaly

3.5 Discover & Explore Cards

In addition to recommendation cards (targeted action suggestions), the AnalyticsEngine generates two additional AI-curated content card types:

- DiscoverCard — surfaces topics or items globally trending on the platform that the user has not yet explored
- ExploreCard — suggests directional expansions: 'based on your activity in X, you should explore Y and its related areas'

Both card types are generated by sampling the RecommendationEngine's content-based and collaborative signals and returning them as typed card objects that the feed and dashboard systems can render independently.

4. TelemetryEngine

4.1 Purpose

The TelemetryEngine is the ingestion and normalization layer. It receives raw PlatformDataEnvelopes — the standard data container for all inter-component messages — normalizes source/target component names, auto-synthesizes DataFlowHops if not provided, routes metrics into the AnalyticsEngine, writes a DataFlowLedgerEntry for every envelope, and returns an EngineFlowSnapshot after each ingest.

4.2 PlatformDataEnvelope

id	UUID for this envelope
flowId	Logical flow identifier grouping related envelopes (e.g. 'flow-kernel-host')
source	Sending component (normalized: 'kogi-kernel', 'gateway', 'service.office', etc.)
target	Receiving component (normalized)
topic	Semantic routing key (e.g. 'kernel.component.registered', 'office.analytics.snapshot')
payload	Map[String, String] — carries metrics, deltas, IDs, and context fields
timestampMs	Event origination time in epoch milliseconds
hops	List[DataFlowHop] — tracing path through platform components; auto-synthesized if empty

4.3 Payload Field Conventions

The TelemetryEngine inspects well-known payload keys to route data to the correct sub-systems:

Payload Key	Routes To	Description
latency_ms · queue_depth · cpu_pct · memory_mb	AnalyticsEngine.ingestModuleMetric	Module performance metrics
host_cpu_pct · host_memory_pct · host_disk_pct · host_process_count	AnalyticsEngine.ingestHostMetric	Host infrastructure metrics
productivity_delta · cashflow_delta · collaboration_delta · risk_delta	AnalyticsEngine.ingest(StreamEvent)	Portfolio signal deltas
profile_id · user_id · item_id · interaction_type · rating	RecommendationEngine.recordInteraction	User behavior for rec feedback loop
event_id · event_type · module	StreamEvent construction	Stream event classification

4.4 DataFlowLedgerEntry

Every ingested envelope produces a ledger entry, providing a full audit trail of all data flows through the platform. The ledger stores the last 50,000 entries in memory (configurable) and is accessible via `KogiEngine.flowLedger(limit)` and the CLI/gRPC snapshot commands.

4.5 ComponentFlowStats

The `TelemetryEngine` computes per-component flow statistics over a configurable rolling time window (default 5 minutes):

- `ingressCount` — number of envelopes targeting this component
- `egressCount` — number of envelopes sourced from this component
- `avgLatencyMs` — average latency extracted from matching payload fields
- `errorCount` — envelopes where `event_type` contains 'error', 'fail', or 'incident'
- `lastTopic / lastSeenMs` — most recent activity for health monitoring

5. DataStreamingEngine

5.1 Purpose

The DataStreamingEngine provides an in-memory append-only event stream with pub/sub subscription support. It is the low-level event bus used internally by the AnalyticsEngine and can also be used directly for any module that needs a lightweight stream with real-time callbacks.

5.2 API

Method	Description
ingest(event: StreamEvent)	Append a single event; delivers to all active subscribers; trims to maxEvents (default 10,000)
ingestBatch(events)	Ingest multiple events in sequence
recent(limit)	Return the last N events from the stream
all	Return the complete stream buffer
eventsByProfile(profileId, limit)	Filter stream by profileId
eventsByModule(module, limit)	Filter stream by module name
eventsSince(sinceMs)	All events with timestamp >= sinceMs
eventsByModuleSince(module, sinceMs)	Combined module + time filter
subscribe(handler)	Register a callback invoked synchronously on every ingest; returns an unsubscribe thunk

The pub/sub model is synchronous and in-process. For production deployments requiring durable, distributed streaming, the DataStreamingEngine should be backed by Kafka topics, with subscriber callbacks replaced by Kafka consumer groups.

6. RecommendationEngine

6.1 Purpose & Architecture

The RecommendationEngine implements a full multi-method recommendation system with user profiling, collaborative filtering, content-based filtering, hybrid scoring, contextual adaptation, cold-start handling, and a continuous feedback loop. It is the intelligence core behind the platform's feed, discover, marketplace matching, and community space recommendation surfaces.

6.2 User Profile & Persona System

Every user is represented by a UserProfile that accumulates interaction history and derives an implicit preference model:

userId	Platform user identifier
demographics	Age range, location, language, occupation
preferences	Map[String, Double] — category/tag preference scores
preferenceVector	Normalized tag → weight map used for content-based scoring
interactionHistory	All UserInteractions (clicks, views, purchases, dismissals, etc.)
feedbackHistory	FeedbackRecord list — tracks recommendation exposure and response
dominantCategories	Top categories by interaction weight — updated after every ingest
sessionContext	Last seen device, location, time-of-day, referrer
persona	Derived PersonaLabel (see below)
createdMs / updatedMs	Profile lifecycle timestamps

6.3 Persona Labels

The persona system classifies users into behavioral archetypes used to tune scoring, UI adaptation, and risk/optimization recommendations across all sub-engines:

Persona	Behavioral Profile	Engine Impact
PowerUser	High interaction frequency, uses advanced features, broad catalogue exploration	Compact layout, RecencyFirst ordering, tighter risk thresholds, higher query row cap
CasualBrowser	Low engagement, short sessions, follows trending content	Spacious layout, TrendingFirst ordering, wider risk tolerance, low query row cap
Explorer	Diverse topic exploration, high discovery/explore engagement	RelevanceFirst ordering, widened discovery pipeline, collaboration signal boost
EarlyAdopter	Rapid adoption of new items, beta features, experimental content	RecencyFirst ordering, high novelty score boost

ValueSeeker	Price/ROI sensitive, cashflow-focused, deal-oriented	ValueFirst ordering, cashflow signal amplification
Specialist	Deep focus in one or few categories, low breadth but high depth	RelevanceFirst, domain-specific recs, higher query row cap, analytic hints
Collaborator	High social engagement, shares, comments, group activities	PersonalFirst ordering, collaboration signal boost
Newcomer	Low history, cold-start, default behavior	Spacious layout, TrendingFirst, cold-start recommendations

6.4 Filtering Methods

Collaborative Filtering

Finds users with similar interaction histories using cosine similarity on preference vectors. Items liked by similar users that the target user has not interacted with are surfaced, weighted by the similarity score and the collaborating user's interaction weight.

Content-Based Filtering

Compares the user's preference vector against item feature vectors (derived from tags, category, and attributes). Items with the highest cosine similarity to the user's profile are returned, with a recency boost applied to recently created items.

Hybrid Model

Blends collaborative (40%), content-based (40%), and popularity (20%) scores with a configurable cold-start fallback. Contextual boosting is applied on top of the blend based on the current InteractionContext (device, location, time-of-day, referrer).

Cold-Start Handling

For new users with insufficient history, recommendations are generated from global popularity scores, category defaults, trending content, and any available demographic signals. The system transitions from cold-start to full hybrid mode as interaction history accumulates.

6.5 Interaction Types & Weights

InteractionType	Category	Signal Weight
Purchase	explicit	5.0
Rate	explicit	4.0
Review	explicit	4.0
Bookmark	explicit	3.0
Share	explicit	3.0
Click	implicit	2.0
View	implicit	1.5
Dwell	implicit	1.0
Search	implicit	1.0

Ignore	implicit negative	-1.0
Dismiss	explicit negative	-2.0

7. PersonalizationEngine

7.1 Purpose

The PersonalizationEngine extends the RecommendationEngine with platform-level personalization concerns: how content is displayed, how users are segmented into cohorts, how A/B experiments are assigned and tracked, how the multi-armed bandit selects between content variants, and how the persona lifecycle is managed over time.

7.2 Content Delivery Adaptation

For each user, the PersonalizationEngine resolves a ContentDeliveryConfig based on their persona, explicit preferences, and interaction context:

Config Field	PowerUser	CasualBrowser / Newcomer	Explorer / Specialist
density	Compact	Spacious	Standard
ordering	RecencyFirst	TrendingFirst	RelevanceFirst
maxFeedItems	50	20	40
showAdvancedFilters	true	false	true (Specialist)
showAnalyticsSidebar	true	false	false
autoExpandItems	false	true	false
highlightNewContent	true	false	true (EarlyAdopter)
suppressLowEngagement	false	true	false

7.3 User Segmentation

Segments are rule-based cohorts defined by attribute key → allowed values maps. A user matches a segment if all rules are satisfied. The PersonalizationEngine assigns users to segments and detects drift (when a user's attributes have moved out of their current segment's rules), triggering re-assignment.

7.4 A/B Experiment Framework

The experiment scaffolding supports:

- Experiment definition — id, name, variants (each with a traffic allocation weight), status (draft / active / paused / completed)
- Variant assignment — deterministic hash-based assignment ensures a user always gets the same variant
- Exposure tracking — records which users were exposed to which variants
- Conversion recording — records when an exposed user completes the target action
- Results summary — exposure count, conversion count, conversion rate, lift calculation per variant

7.5 Multi-Armed Bandit (UCB1)

For content slot optimization where the best variant is unknown, the PersonalizationEngine implements the UCB1 (Upper Confidence Bound) algorithm:

```
UCB1 score = average_reward + sqrt(2 * ln(total_pulls) / arm_pulls)
```

```
Arms with zero pulls are selected first (exploration priority).  
After sufficient pulls, exploitation naturally dominates.
```

7.6 Persona Lifecycle

Personas are not static labels — they evolve as user behavior changes:

- build — initial persona inference from interaction history and demographics
- confirm — persona is stable for N consecutive sessions without drift
- drift detection — compares current interaction pattern against persona archetype thresholds
- update — reassigns persona label if drift exceeds configured threshold
- history — maintains full persona change log for auditability and trend analysis

7.7 Dynamic Content Adaptation Rules

A rule table maps (PersonaLabel × ContentOrdering × ContentDensity) to template configurations. These rules can be overridden by explicit user preferences, which always take precedence over inferred defaults.

8. MatchEngine

8.1 Purpose

The MatchEngine performs multi-dimensional scoring to match any MatchSubject against any other MatchSubject or set of subjects. It is the foundation for worker-to-opportunity matching, investor-to-campaign matching, collaborator discovery, resource-to-project matching, and analytics artifact recommendation.

8.2 Subject Type Hierarchy

MatchSubject	Type	Key Discriminants
UserSubject	User	role (Owner/Investor/Donor/Contributor/...) · persona · skills · interests · budget · location
ComponentSubject	Portfolio component	kind (Portfolio/Program/Project/Resource/Artifact/Asset) · status · requiredSkills · budget · tags
ResourceSubject	Resource/asset	resourceType · capacity · availability · cost · location · tags
AnalyticsArtifact	Rec/search/index	artifactType (Recommendation/Search/Index/Filter) · affinity tags · score

8.3 Scoring Dimensions

Every match pair is scored across four dimensions, combined via configurable weights:

Dimension	Weight (default)	Computation
Tag similarity	40%	Jaccard similarity of subject tag sets
Attribute affinity	30%	Overlap of shared attribute key-value pairs
Persona alignment	20%	Persona-to-component type affinity matrix (e.g. PowerUser ↔ AssetKind = high)
Signal strength	10%	Analytics signal from interaction history and engagement metrics

8.4 Key Match Operations

Operation	Input → Output	Use Case
matchProfilesToResources	Seq[Profile] × Seq[Resources] → Map[Profile, Seq[Resources]]	Assign best resources to worker profiles
matchUsersToComponents	Seq[UserSubject] × Seq[ComponentSubject] → ranked pairs	Find best contributors / owners for projects

matchInvestorsToCampaigns	Seq[UserSubject(Investor)] × campaigns → ranked pairs	Investment opportunity matching
matchCollaborators	UserSubject × pool → ranked candidates	Find collaborators for a project or space
matchSkillsToRoles	skill requirements × UserSubject skills → coverage score	Staffing plan fulfillment
matchAnalyticsArtifacts	query intent × AnalyticsArtifact pool → ranked artifacts	Surface relevant recommendations, indexes, and filters

8.5 Persona-Aware Boosting

The MatchEngine applies persona-based boosts to the final score. For example:

- PowerUser matched to AssetKind or ProgramKind components receives a 1.2× multiplier
- Investor role matched to campaigns with equity instrument receives a 1.3× multiplier
- Explorer persona receives a diversity bonus that surfaces less-obvious but high-potential matches
- Specialist persona receives a depth bonus for domain-exact tag matches

9. GraphEngine

9.1 Purpose

The GraphEngine operates on the Kogi portfolio dependency graph — a directed, typed, weighted property graph representing the relationships between all portfolio components (portfolios, programs, projects, resources, assets, artifacts) and the platform topology. It provides dependency closure, impact analysis, cycle detection, topological scheduling, critical path analysis (CPM), and snapshot diffing.

9.2 Graph Data Model

Type	Fields	Meaning
GraphNode	id · duration	A vertex; duration is used in CPM (task duration in time units)
GraphEdge	from · to · edgeType · weight	A directed typed edge; weight is edge traversal cost for CPM
EdgeType	Dependency Hierarchy Relationship	Dependency = must-complete-before; Hierarchy = parent/child; Relationship = general link

9.3 Graph Algorithms

Dependency Closure & Reverse Closure

Forward DFS on Dependency edges: returns all nodes this node transitively depends on. Reverse DFS: returns all nodes that transitively depend on this node. Together, these power the ImpactReport — 'if I change X, what is affected downstream, and what depended on it upstream?'

Cycle Detection

DFS graph colouring (0 = unvisited, 1 = in-stack, 2 = done). Back-edges indicate cycles. The algorithm reconstructs the full cycle path by tracing the parent map from the cycle endpoint back to the cycle start. Returns CycleReport(hasCycles, Seq[Seq[String]]).

Topological Sort (Kahn's Algorithm)

BFS in-degree reduction. Nodes with zero in-degree are enqueued first; as nodes are processed their successors' in-degrees are decremented. If the result contains fewer nodes than the graph, the remaining nodes form cycle sets (returned as Left(cycleNodes)).

Critical Path Analysis (CPM)

```
Forward pass:  EST(node) = max(EST(predecessor) + edge.weight)  for all incoming
edges
               EFT(node) = EST(node) + node.duration

Backward pass: LFT(node) = min(LFT(successor) - edge.weight)    for all outgoing
edges
               LST(node) = LFT(node) - node.duration

Slack(node)   = LST(node) - EST(node)
```

```
Critical path = nodes where Slack < 1e-9 (floating-point tolerance)
```

Graph Diffing

Compares two GraphEngine instances and returns a GraphDiff identifying: added nodes, removed nodes, purely added edges, purely removed edges, and changed edges (same from/to endpoints but mutated weight or type). Used for portfolio change tracking and event-sourced state reconstruction.

9.4 GraphEngineAPI Facade

GraphEngineAPIFacade.scala provides a fluent, type-safe Scala API wrapping GraphEngine with:

- Builder pattern (GraphEngineBuilder) for addEdge / addNode / addDependency / addHierarchy
- Either-typed results: Right(result) on success, Left(NodeNotFound | CyclePresent | EngineError) on failure
- Mutation methods (withEdge, withoutEdge, withoutNode) returning new API instances (immutable-style)
- Bulk helpers: allImpacts(), roots(), leaves(), inDegrees(), outDegrees(), byImpactSize()
- Snapshot and diffing: snapshot() returns an opaque Snapshot; diff(earlier) computes what changed

9.5 GraphEngineAPIServer (HTTP REST)

GraphEngineAPIServer.scala exposes the GraphEngine via a lightweight HTTP server (JDK HttpServer):

Endpoint	Method	Description
GET /health	GET	Health check — returns {status: ok, engine: graph-engine}
POST /load	POST	Load a new graph from JSON body {edges: [...], nodes: [...]}
GET /nodes	GET	List all node IDs in the current graph
GET /edges	GET	List all edges as {from, to, type, weight} JSON array
GET /impact/{nodeId}	GET	ImpactReport for a node: affected + dependents sets
GET /topo	GET	Topological order of dependency graph; 409 if cycle present
GET /critical	GET	Critical path report: path, totalCost, nodeSlack map
GET /cycles	GET	Cycle detection: hasCycles flag + cycle path arrays
POST /diff	POST	Diff current graph against a JSON-encoded snapshot

10. RiskEngine

10.1 Purpose

The RiskEngine scores portfolio health from a PortfolioSignal, produces status verdicts (green / amber / red), and generates optimization plans with prioritized recommendations. It is used by the AnalyticsEngine to annotate every snapshot and by the KogiEngine directly for standalone risk assessment calls.

10.2 Scoring Model

```
health_score =
  productivity  × 0.30
+ cashFlow    × 0.30
+ collaboration × 0.20
+ (100 - risk) × 0.20

Status: green  if score >= 80
      amber   if score >= 60
      red     if score < 60
```

10.3 Persona-Aware Scoring

The RiskEngine adjusts signal weights based on persona before scoring, reducing alert fatigue for low-engagement users and tightening thresholds for power users:

Persona	Signal Adjustment
PowerUser	risk × 1.10 — tighter risk guardrails
CasualBrowser	risk × 0.85, productivity × 0.90 — reduced noise
Explorer	collaboration × 1.15 — compensates for naturally lower collaboration score
ValueSeeker	cashFlow × 0.90 — amplifies cashflow sensitivity

10.4 Risk Optimization Thresholds

Condition	Priority	Recommendation
risk >= 70.0	high	Stabilize exchange, wallet, and compliance workflows
cashFlow < 50.0	high	Tighten receivables and review overdue invoices
productivity < 50.0	medium	Rebalance sprint workload, reduce WIP
collaboration < 50.0	medium	Schedule cross-team reviews, unblock approvals
health.status == red	high	Run stabilization playbook before new commitments

11. OptimizationEngine

11.1 Purpose

The OptimizationEngine analyzes platform workload metrics (CPU, memory, queue depth, latency) and produces an OptimizationPlan with scored recommendations for scaling, caching, concurrency, and hot-path tuning. Recommendations are persona-aware, so the same metrics produce different suggestions depending on the user's behavioral profile.

11.2 Scoring

```
optimization_score =
  (100 - cpu_pct)           × 0.30
+ (100 - memory_pct)       × 0.25
+ (100 - queue_depth / 2)  × 0.20
+ (100 - latency_ms / 20)  × 0.25

All sub-scores clamped to [0.0, 100.0]
```

11.3 System Thresholds

Metric	Threshold	Priority	Recommendation
cpuPct	>= 80%	high	CPU saturation: consider horizontal scaling or workload shedding
memoryPct	>= 80%	high	Memory pressure: tune caches, reclaim inactive workers
queueDepth	>= 150	medium	Elevated queue: increase consumer concurrency or introduce backpressure
latencyMs	>= 1200	medium	Latency p95 above threshold: identify slow handlers, optimize hot paths

11.4 Persona-Aware Recommendations

Persona	Trigger Condition	Additional Recommendation
PowerUser	latencyMs > 800	Prioritize p95 latency reduction over memory savings
CasualBrowser	cpuPct > 60	Defer background jobs during low-engagement windows
Explorer	queueDepth > 80	Widen discovery pipeline concurrency for broad content requests

12. SearchEngine

12.1 Purpose

The SearchEngine provides full-text search, tag filtering, metadata filtering, and personalized result re-ranking over an in-memory document index (max 50,000 documents, configurable). It serves all search surfaces: portfolio item search, space discovery, worker talent search, and marketplace listing search.

12.2 SearchDocument & SearchQuery

SearchDocument.id	Unique document identifier
SearchDocument.title	Primary text field (highest weight in scoring)
SearchDocument.body	Secondary text content
SearchDocument.tags	Tag labels for exact-match boost (0.5 per matching tag)
SearchDocument.metadata	Map[String, String] — arbitrary key/value for filtered queries
SearchQuery.text	Free-text query — tokenized, case-folded, stemmed to words
SearchQuery.tags	Required tags — document must contain ALL listed tags
SearchQuery.metadata	Required metadata — document must match ALL listed key-value pairs
SearchQuery.limit	Max results returned (default 10)

12.3 Scoring

```
score(doc, query) =
  count(query_tokens ∩ doc_text_tokens)           // full-text token hits
  + |query.tags ∩ doc.tags| × 0.5                  // tag intersection boost

Results sorted descending by score; zero-scoring docs excluded.
```

12.4 Personalized Search

The personalizedSearch(PersonalizedSearchQuery) method re-ranks base results using the user's preference vector and dominant categories:

```
personalized_score(doc) =
  base_score(doc)
  + Σ (preferenceVector[tag] for tag in doc.tags) × 0.4 // preference boost
  + 1.5 if doc.metadata["category"] ∈ dominantCategories // category boost
```


13. QueryEngine

13.1 Purpose

The QueryEngine interprets, normalizes, analyzes, and optimizes SQL queries before execution against the Kogi PostgreSQL database. It provides cost estimation, safety warnings (SELECT *, missing WHERE, full-table DELETE/UPDATE), index hints, and a persona-aware row cap system that prevents accidental full-table scans by casual users while allowing power users and specialists full analytical access.

13.2 Pipeline



13.3 Persona-Aware Row Caps

Persona	Auto LIMIT	Analytic Hints
PowerUser	10,000 rows	Enabled — multi-join materialized view suggestions, async execution hints
Specialist	5,000 rows	Enabled
CasualBrowser	200 rows	Disabled — simplified hints only
Default / Other	profile.preferredLimit (default 1,000)	Disabled unless allowAnalyticHints = true

14. gRPC Interface — EngineGrpcServer

14.1 Service Definition

The gRPC server uses `google.protobuf.Struct` as the universal wire format for both request and response messages, enabling schema-free operation without `.proto` file regeneration for each new command type. The service is registered at `kogi.engine.v1.EngineService`.

14.2 RPC Methods

Method	Request Fields	Response Fields
Control	action (start pause stop resume)	engine · status · mode · changed · timestamp_ms
Status	(empty)	engine · status · mode · changed · timestamp_ms
Ingest	topic · source · target · flow_id · timestamp_ms · payload (struct)	engine · status · total_envelopes · observed_topics · generated_at_ms · host{...} · recommendations[...]
Snapshot	host_id · window_ms	Same as Ingest response — full EngineFlowSnapshot

14.3 Codec

EngineGrpcCodec provides bidirectional conversion between `Scala Map[String, Any]` and `google.protobuf.Struct`. Supported value types: `String`, `Boolean`, `Int`, `Long`, `Double`, `Float`, `Map[String, Any]` (nested Struct), `Seq[_]` (`ListValue`). Unrecognized types fall back to `.toString`.

14.4 Configuration

Port	Default 9100; overridable via <code>KOGI_ENGINE_GRPC_PORT</code> environment variable
Reflection	<code>ProtoReflectionService</code> registered — allows gRPC UI and <code>grpcurl</code> introspection
Shutdown	JVM shutdown hook calls <code>server.shutdown()</code> for graceful drain
Threading	Default Netty thread pool; tune via gRPC server builder if needed

15. CLI Interfaces

15.1 KogiEngineCli — Engine CLI

KogiEngineCli provides a command-line interface to the full KogiEngine, emitting JSON responses to stdout. Designed for integration with shell scripts, Go service health checks, and local debugging.

Command	Key Arguments	Action
control	--action start pause stop	Transition engine lifecycle mode
status	(none)	Print current mode and timestamp
snapshot	--host-id --window-ms	Full EngineFlowSnapshot as JSON
ingest	--topic --source --target --payload key=value...	Ingest a gateway message and return snapshot
query	--sql "..."	Parse, analyze, and optimize a SQL string
search	--text --tags --limit	Run a search query against the index
graph-impact	--node	ImpactReport for a node
graph-cycles	(none)	CycleReport across all edges
graph-critical	(none)	Critical path report
graph-topo	(none)	Topological sort order
graph-diff	--edge from:to:type...	Diff current graph against supplied edge list

15.2 GraphEngineCLI — Graph-Specific CLI

GraphEngineCLI is a standalone CLI exclusively for the GraphEngine, with CSV/DOT export support for integration with visualization tools (Graphviz, Gephi, etc.):

Command	Description
impact --node ID	Full ImpactReport: affected + dependents sets
closure --node ID	Forward dependency closure (transitive deps)
rdeps --node ID	Reverse dependency closure (transitive dependents)
topo	Topological sort; exits 1 on cycle
critical	Critical path with slack values per node
cycles	All detected cycles with full path reconstruction
neighbors --node ID	All nodes reachable from this node in any direction
ancestors --node ID	Hierarchy ancestors
descendants --node ID	Hierarchy descendants
diff --edges A:B:dep,C:D:hier	Diff current graph against an alternative edge set

nodes	List all node IDs
edges	List all edges
export --format csv dot json	Export graph in CSV (edge list), DOT (Graphviz), or JSON format

Both CLIs support --format json|text output. JSON mode is suitable for programmatic consumption by Go services; text mode is optimized for human-readable debugging and monitoring.

16. Complete Data Model Reference

16.1 Core Event & Signal Types

Type	Package Role	Key Fields
StreamEvent	AnalyticsEngine input	profileId · module · eventType · productivity/cashFlow/collaboration/riskDelta
ModuleMetricSample	AnalyticsEngine input	module · latencyMs · queueDepth · errorCount · throughputUnits · cpuPct · memoryMb
HostMetricSample	AnalyticsEngine input	hostId · cpuPct · memoryPct · diskPct · processCount · schedulerLoad · network
PlatformDataEnvelope	TelemetryEngine input	id · flowId · source · target · topic · payload · timestampMs · hops
DataFlowHop	Envelope tracing	component · stage · timestampMs · metadata
PortfolioSignal	Risk/Analytics shared	productivity · cashFlow · collaboration · risk (all 0–100)
UserInteraction	RecommendationEngine	userId · itemId · interactionType · value · sessionId · context
ItemProfile	RecommendationEngine	id · title · category · tags · attributes · popularityScore
UserProfile	RecommendationEngine	userId · interactionHistory · preferenceVector · dominantCategories · persona
GraphEdge	GraphEngine	from · to · edgeType(Dependency Hierarchy Relationship) · weight
GraphNode	GraphEngine	id · duration (CPM task duration)
MatchSubject	MatchEngine	sealed: UserSubject ComponentSubject ResourceSubject AnalyticsArtifact

16.2 Output / Result Types

Type	Producer	Contents
AnalyticsSnapshot	AnalyticsEngine	per-profile: signal · health · module activities · rec/discover/explore cards
SystemRealtimeSnapshot	AnalyticsEngine	host snapshot + module snapshots + system-level AI cards
EngineFlowSnapshot	TelemetryEngine	system snapshot + component flow stats + ledger + observed topics + total envelopes
ImpactReport	GraphEngine	root node + affected set + dependents set
CycleReport	GraphEngine	hasCycles flag + Seq[Seq[String]] cycle paths
CriticalPathReport	GraphEngine	critical path node order + totalCost + nodeSlack map

GraphDiff	GraphEngine	added/removed nodes, added/removed/changed edges
PortfolioHealthScore	RiskEngine	overall score + status (green/amber/red) + recommendations
RiskOptimizationPlan	RiskEngine	health score + riskScore + prioritized recommendations
OptimizationPlan	OptimizationEngine	workload score + prioritized recommendations
QueryPlan	QueryEngine	analysis + optimizedSql + hints + generatedAtMs
SearchResult	SearchEngine	total + List[SearchMatch{document, score, highlights}]
ContentDeliveryConfig	PersonalizationEngine	density + ordering + maxFeedItems + feature flags
PersonaBuildResult	RecommendationEngine	userId + persona label + dominant categories + evidence

17. DEPLOYMENT & INTEGRATION

17.1 Entry Points

Main.scala	Standalone demo / smoke test — ingests 5 platform envelopes, prints snapshot to stdout
EngineGrpcServer	Production gRPC server — listens on KOGI_ENGINE_GRPC_PORT (default 9100)
GraphEngineAPIServer	HTTP REST server for GraphEngine — listens on configurable port (default 8080)
KogiEngineCli	Interactive / scripted CLI for full engine — run with args or pipe from stdin
GraphEngineCLI	Standalone graph analysis CLI — loads CSV edge/node files

17.2 Go Service Integration

The kogi-engine Go service (kogi-services) acts as the bridge between the Go microservice layer and the Scala engine. It communicates via gRPC to EngineGrpcServer:

```
// Go service pattern
conn, _ := grpc.Dial("kogi-engine:9100", grpc.WithInsecure())
client := enginev1.NewEngineServiceClient(conn)

// Ingest a gateway event
resp, _ := client.Ingest(ctx, &structpb.Struct{
    Fields: map[string]*structpb.Value{
        "topic": structpb.NewStringValue("office.portfolio.updated"),
        "source": structpb.NewStringValue("service.office"),
        "target": structpb.NewStringValue("kogi-engine"),
        "payload": structpb.NewStructValue(...),
    },
})

// Get system snapshot
snap, _ := client.Snapshot(ctx, &structpb.Struct{
    Fields: map[string]*structpb.Value{
        "host_id": structpb.NewStringValue("kogi-host-001"),
        "window_ms": structpb.NewNumberValue(300000),
    },
})
```

17.3 Scaling Considerations

- In-memory state: all sub-engines currently use in-memory storage. Production deployment should back StreamEvents and envelopes with Kafka, user profiles with Redis, and document indices with Elasticsearch.
- gRPC concurrency: the Netty-based gRPC server handles concurrent requests on its thread pool. Engine state mutations are not currently thread-safe — wrap KogiEngine in a synchronized facade or use actor-based concurrency (Akka) for high-throughput deployments.

- GraphEngine rebuild cost: every graph mutation rebuilds the full adjacency maps from the edge/node Seq. For large graphs (>10K nodes), consider a mutable adjacency map with incremental updates.
- Multi-host federation: each kogi-host can run an independent engine instance. Cross-host synchronization of recommendation profiles and graph state should use the Kafka event bus and CRDT-based merge strategies for distributed convergence.

APPENDIX — QUICK REFERENCE

Sub-engine Configuration Defaults

Parameter	Default	Sub-engine
DataStreamingEngine.maxEvents	10,000	DataStreamingEngine
AnalyticsEngine.defaultWindow	250 events	AnalyticsEngine
AnalyticsEngine.defaultRealtimeWindowMs	5 minutes	AnalyticsEngine
AnalyticsEngine.recommendationLimit	8	AnalyticsEngine
AnalyticsEngine.discoverLimit	8	AnalyticsEngine
AnalyticsEngine.exploreLimit	8	AnalyticsEngine
AnalyticsEngine.maxModuleMetrics	20,000	AnalyticsEngine
AnalyticsEngine.maxHostMetrics	20,000	AnalyticsEngine
TelemetryEngine.maxEnvelopes	50,000	TelemetryEngine
TelemetryEngine.maxLedger	50,000	TelemetryEngine
SearchEngine.maxDocuments	50,000	SearchEngine
QueryEngine default row cap	1,000 rows	QueryEngine
QueryEngine PowerUser cap	10,000 rows	QueryEngine
EngineGrpcServer port	9100	EngineGrpcServer
TelemetryEngine snapshot window	5 minutes	TelemetryEngine

Known Modules (AnalyticsEngine.knownModules)

kogi-kernel · kogi-host · kogi-server · gateway · auth · portfolio · exchange · ims · office · bank · studio · community · marketplace · engine · network · database